

Data Mining – Minimal Code Sheet (from lectures)

Short, simple, ready-to-memorize snippets.

Notes

- Replace filenames (`data.csv`, `basket_onehot.csv`) with yours.
- Apriori/FP-Growth require: `pip install mlxtend`.

0) Imports (common)

```
import numpy as np
import pandas as pd
```

1) Missing Values (pandas)

```
df = pd.read_csv("data.csv")
df.isnull().sum()           # count NaNs per column

df.fillna(0)                # fill NaN with 0

df.fillna(method="ffill")   # forward fill
df.fillna(method="bfill")   # backward fill

df.interpolate()           # linear interpolation (numeric)
```

2) Normalization

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
X = pd.read_csv("data.csv")           # numeric columns only
X_mm = MinMaxScaler().fit_transform(X) # min-max [0,1]
X_z = StandardScaler().fit_transform(X) # z-score
```

3) PCA (dimensionality reduction)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X = pd.read_csv("data.csv")
X = StandardScaler().fit_transform(X)
X_pca = PCA(n_components=2).fit_transform(X)
```

4) Feature Selection (SelectKBest)

```
from sklearn.feature_selection import SelectKBest, chi2

df = pd.read_csv("diabetes.csv")
X = df.drop(columns=["Outcome"])
y = df["Outcome"]
X_sel = SelectKBest(chi2, k=4).fit_transform(X, y)
```

5) Association Rules (Apriori) – mlxtend

```
from mlxtend.frequent_patterns import apriori, association_rules

basket = pd.read_csv("basket_onehot.csv") # one-hot (0/1 or True/False)
freq = apriori(basket, min_support=0.3, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=0.7)
rules[["antecedents", "consequents", "support", "confidence", "lift"]].head()
```

6) FP-Growth – mlxtend

```
from mlxtend.frequent_patterns import fpgrowth, association_rules

basket = pd.read_csv("basket_onehot.csv")
freq = fpgrowth(basket, min_support=0.3, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=0.7)
rules[["antecedents", "consequents", "support", "confidence", "lift"]].head()
```

7) K-Means (clustering)

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(pd.read_csv("data.csv"))
lab = KMeans(n_clusters=3, random_state=0, n_init="auto").fit_predict(X)
lab
```

8) Hierarchical (Agglomerative)

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(pd.read_csv("data.csv"))
lab = AgglomerativeClustering(n_clusters=3,
```

```
linkage="single").fit_predict(X) # single/complete/average
lab
```

9) DBSCAN (density + noise/outliers)

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(pd.read_csv("data.csv"))
lab = DBSCAN(eps=0.5, min_samples=5).fit_predict(X)
# -1 means noise/outlier
lab
```

10) EM (Gaussian Mixture Model)

```
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(pd.read_csv("data.csv"))
gmm = GaussianMixture(n_components=3, random_state=0).fit(X)
lab = gmm.predict(X) # cluster labels
probs = gmm.predict_proba(X) # membership probabilities
lab, probs[:3]
```

11) Outlier Detection (common sklearn)

```
from sklearn.ensemble import IsolationForest

X = pd.read_csv("data.csv")
out = IsolationForest(contamination=0.05,
random_state=0).fit_predict(X)
# -1 outlier, 1 inlier
out

from sklearn.neighbors import LocalOutlierFactor

X = pd.read_csv("data.csv")
out = LocalOutlierFactor(n_neighbors=20,
contamination=0.05).fit_predict(X)
out
```

12) Mahalanobis + Chi-square threshold (lecture-style)

```
import numpy as np
import pandas as pd
from scipy.stats import chi2

X = pd.read_csv("data.csv").values
mu = X.mean(0)
```

```
Sinv = np.linalg.inv(np.cov(X, rowvar=False))  
d2 = np.sum((X-mu) @ Sinv * (X-mu), axis=1) # squared Mahalanobis  
thr = chi2.ppf(0.95, df=X.shape[1])  
outliers = d2 > thr  
outliers
```